PROGRAMACION ESTRUCTURADA BASICA

UNIDAD 1.

Algoritmos sobre Vectores

Material preparado por: Dr. Ing. Pablo Martín Vera

INDICE

Contenido

1. NÚMEROS ALEATORIOS 2
2. MÁXIMOS Y MÍNIMOS MÚLTIPLES 4
3. BÚSQUEDA SECUENCIAL EN VECTORES 5
4. CARGA DE UN VECTOR SIN ADMITIR VALORES REPETIDOS 6
5. ORDENAMIENTO DE VECTORES 7
5.1 ORDENAMIENTO POR SELECCIÓN 7
5.2 ORDENAMIENTO POR BURBUJEO 10
5.3 ORDENAMIENTO POR BURBUJEO OPTIMIZADO 12
5.4 ORDENAMIENTO DE VECTORES PARALELOS 13
6. BÚSQUEDA BINARIA 14

OBJETIVOS: *Aplicar distintos algoritmos como búsqueda, ordenamiento, máximos y mínimos, etc sobre los vectores para permitir resolver problemas complejos.*

1. Números Aleatorios

Para probar algunos de los algoritmos que se verán en esta unidad y no tener que realizar una carga manual de los datos, en ocasiones podemos recurrir a la generación de números aleatorios para poder generar un lote de prueba rápidamente y distinto cada vez que se ejecuta el programa.

En realidad, el lenguaje no tiene la capacidad de generar números al azar en forma directa, sino que genera números distintos en base a un valor de partida lo que se denomina números pseudoaleatorios ya que si no se cambia el número de partida en las distintas ejecuciones del programa la secuencia de números siempre será la misma.

Entonces el primer paso es establecer dicho valor de partida de una forma que cambie cada vez que se inicia el programa y eso se puede hacer recuperando la fecha y hora actual.

Para ello se utilizará la función de la biblioteca time.h denominada time y se establece el valor inicial de los números pseudoaleatorios.

El procedimiento es el siguiente:

1) Incorporar la biblioteca
   #include <time.h>

2) Establecer el valor inicial con la función srand enviando la fecha y hora actual.
   srand(time(NULL);

3) Recuperar un número aleatorio usando la función rand()

El siguiente código muestra un número distinto cada vez que se ejecuta el programa:

```
#include <stdio.h>
#include <time.h>
int main()
{
    int num;
    srand(time(NULL));
    num = rand();
    printf("El numero al azar es: %d", num);
    return 0;
}
```

En este caso los números serán de 5 cifras, pero si quiere obtener número de menor cantidad de cifras simplemente se le puede aplicar el resto de la división entera. El siguiente ejemplo carga de forma aleatoria un vector de 10 elementos con números de máximo 2 cifras.

```
#include <stdio.h>
#include <time.h>
#define TAM 10
void CargaAleatoria(int[],int, int);
void Mostrar(int[],int);
int main()
{
    int vec[TAM];
    srand(time(NULL));
    CargaAleatoria(vec,TAM,100);
    Mostrar(vec,TAM);
    return 0;
}

void CargaAleatoria(int v[],int ce, int max)
{
    int i;
    for (i=0;i<ce;i++)
        v[i]=rand()%max;
}
void Mostrar(int v[],int ce)
{
    int i;
    for (i=0;i<ce;i++)
        printf("%d\n", v[i]);
}
```

2. Máximos y Mínimos Múltiples

Dado un conjunto de datos, muchas veces es necesario recuperar el valor más grande o el más chico del lote, para ello se realizará un algoritmo donde el primer elemento del conjunto es tomado como referencia y luego comparado con el resto. Como todos los datos ya están todos almacenados en una colección se podrá no solo determinar el mayor valor del conjunto, sino también determinar cuántas veces se repite y en que posiciones de la colección se encuentra.

El procedimiento consta de dos pasos:

1. Determinar el máximo o el mínimo valor del conjunto de datos
2. Volver a recorrer el conjunto de datos para ver cuántas veces se repite y donde se encuentra

El siguiente programa permite ingresar por teclado un vector de 10 elementos utilizando una función de carga completa, luego determina el valor máximo mediante una función y por último informa mediante otra función, la cantidad de repeticiones y las posiciones del máximo.

```
#include <stdio.h>
void Carga(int[], int);
int Maximo(int[], int);
void MostrarMaximo(int[],int,int);

int main()
{
    int vec[10], max;
    Carga(vec,10);
    max = Maximo(vec,10);
    MostrarMaximo(vec,max,10);
    return 0;
}


void Carga(int v[], int N)
{
    int i;
    for (i=0;i<N;i++)
    {
        printf("Ingrese un numero (%d): ",i+1);
        scanf("%d",&v[i]);
    }
}
```

```
int Maximo(int v[], int N)
{
    int max = v[0], i;
    /*se comienza en 1 ya que el primero
    se tomó como referencia*/
    for (i=1;i<N;i++)
    {
        if (v[i]>max)
            max = v[i];
    }
    return max;
}

void MostrarMaximo(int v[],int max,int N)
{
    int cont=0,i;
    printf("El valor maximo es: %d y se encuentra en las \
            siguientes posiciones del vector:\n",max);
    for (i=0;i<N;i++)
    {
        if (v[i]==max)
        {
            printf("%d\n", i+1);
            cont++;
        }
    }
    printf ("El valor maximo se repite %d veces", cont);
}
```

En el ejemplo a las posiciones que se muestran se les suma 1 para que sea más amigable y natural para el usuario para que la primera no sea la posición 0 sino la 1.

3. Búsqueda secuencial en vectores

Frecuentemente es necesario buscar un determinado elemento en un vector, ya sea para determinar si el mismo está en el vector o para recuperar la posición en donde se encuentra.

Los datos del problema son:

- El vector.
- El elemento a buscar.
- La cantidad de elementos sobre los cuales se va a realizar la búsqueda

Las salidas son:

- Un indicador de la existencia del elemento en la colección
- La posición donde se encuentra.

Como la función solo puede retornar un valor, ambas salidas se unifican en una que represente, si existe, la posición donde se encuentra (0 ≤ posición ≤ cantidad de elementos) y si no existe retornará un -1 (valor inválido como posición).

Si bien existen distintos métodos de búsqueda, veremos el más sencillo de ellos que consiste en ir recorriendo uno a uno los elementos del vector hasta que encontremos el buscado. Este método se denomina **Búsqueda Secuencial**.

```
int Buscar(int vector[], int datoABuscar, int cantElem)
{
    int i=0,pos=-1;
    while (pos==-1 && i<cantElem)
    {
        if (vector[i]==datoABuscar)
            pos = i;
        else
            i++;
    }
    return pos;
}
```

Este algoritmo inicia con dos variables la variable i utilizada como índice para moverse por las posiciones del vector y la variable pos que indica la posición donde se encuentra el dato que se está buscando. De entrada, se inicia con un valor -1 que indica que el dato aún no se ha encontrado.

Se realiza un ciclo while ya que no siempre se debe recorrer el vector completo, una vez que se encuentra el dato se sale y no se sigue recorriendo es por ello que el ciclo while tiene dos condiciones:

pos == -1 indica que aún el dato no se ha encontrado
i < cantElem indica que aún quedan datos sobre los cuales seguir realizando la búsqueda

Como ambas condiciones están unidas con el operador AND ambas deben ser verdaderas para ingresar al ciclo entonces ingresa mientras no se haya encontrado el dato buscado y aún quedan dato sobre los cuales realizar la búsqueda.

El ciclo termina si se encuentra el dato al cambiar el valor de pos o cuando no se encuentra pero se llegó al final del vector porque el índice i es igual a la cantidad de elementos y por lo tanto se recorrió todo el vector ya que el primer índice válido es el 0.

La función retorna el contenido de la variable pos que quedará con la posición donde se encontró el dato buscado o -1 si nunca se lo encuentra.

4. Carga de un vector sin admitir valores repetidos

Utilizando la función de búsqueda es posible modificar la carga de un vector asegurándonos de que todos los valores ingresados sean diferentes. Para ello por cada vez que se ingrese un dato se buscará si el mismo ya está en el vector y si está se solicitará un dato diferente.

```
void CargarSinDuplicados(int vector[], int cant)
{
    int i, pos, aux;
    for (i=0; i<cant; i++)
    {
        do
        {
            printf("Ingrese un número:");
            scanf("%d",&aux);
            pos = Buscar(vector, aux, i);
            if (pos!=-1)
                printf("Dato duplicado. Ingrese otro\n");
        }while(pos!=-1);
        vector[i] = aux;
    }
}
```

Puede verse que cada vez que se ingresa un dato, se busca si ya estaba en el vector y si estaba se vuelve a solicitar otro dato que se guarda en la misma posición del vector. No se avanza de posición del vector hasta que no se cargue un dato diferente al resto. La función de búsqueda se llama enviando la variable i como cantidad de datos del vector que indica cuantos se cargaron previamente. La primera vez se envía 0 como tamaño del vector por lo tanto dentro de la función de búsqueda ni siquiera ingresa al ciclo y retorna directamente -1, lo que es correcto ya que al no existir datos previos no puede haber duplicados. La segunda vez ya comparará si el segundo dato ingresado coincide con el primero, y así sucesivamente cada vez se hará la búsqueda con más posiciones del vector que ya fueron cargadas previamente.

5. Ordenamiento de vectores

Los métodos de ordenamiento son numerosos. Podemos considerar dos grandes grupos:
- Directos: Burbujeo, selección e inserción -
- Indirectos (avanzados): Shell, ordenación por mezcla.

En listas pequeñas, los métodos directos se comportan en forma eficiente, mientras que, en vectores con gran cantidad de elementos, se debe recurrir a métodos avanzados.

5.1 Ordenamiento por selección

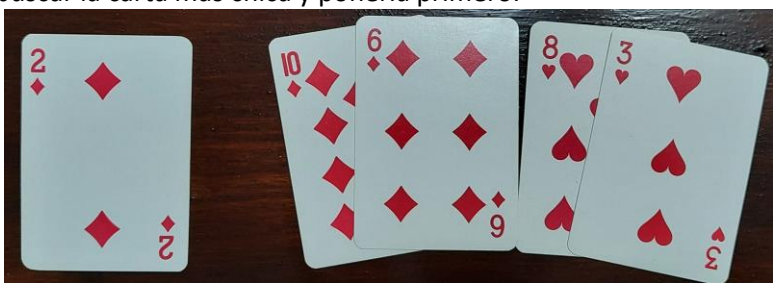
Tomando el siguiente vector de punto partida el objetivo será ordenarlo de menor a mayor.

10	6	2	8	3

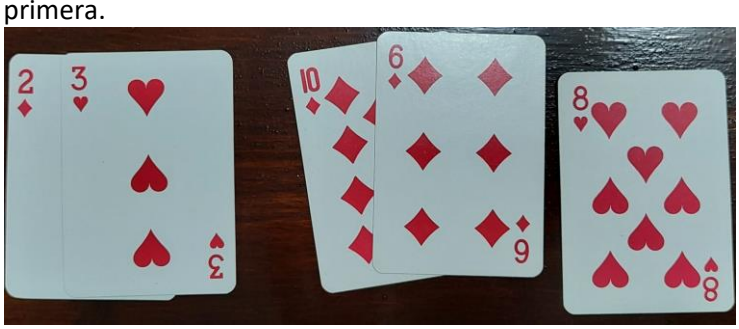
¿Como podríamos ordenarlo haciendo que en la primera posición se ubique el número más chico luego el siguiente y así hasta que queden orden creciente?

¿Para ayudarnos a pensar como lo haríamos imaginemos como haríamos para ordenar el siguiente conjunto de cartas?

Una forma sería buscar la carta más chica y ponerla primero.



Luego de las que quedaron desordenadas nuevamente buscar la mas chica y ponerla a continuación de la primera.



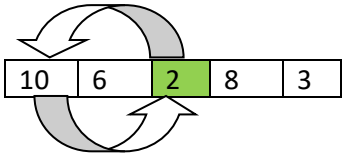
Y así seguir con el mismo procedimiento

Hasta que queden todas ordenadas

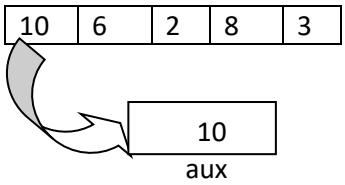
Volvamos ahora al vector que planteamos al comienzo y apliquemos el mismo algoritmo, el primer paso será buscar el menor del vector. En este caso el menor valor es el 2

10	6	2	8	3

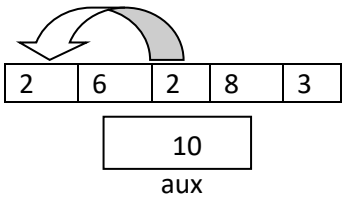
Entonces ese valor tenemos que llevarlo a la primera posición del vector, pero como el valor que se encuentra en la primera posición no tenemos que perderlo lo que vamos a hacer es intercambiarlo con el 2.



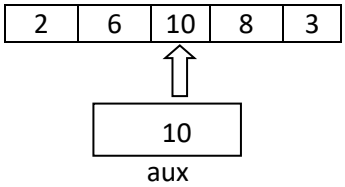
Pero recordemos que la asignación es destructiva no puedo guardar el 2 en la primera posición sin perder el 10, por lo tanto antes hay que copiar el 10 para no perderlo en una variable auxiliar.



Una vez resguardado el 10 podemos copiar el 2 en la primera posición del vector



Y luego donde estaba el 2 asignar lo que tenemos en la variable auxiliar para completar el intercambio



De esta forma ahora el 2 quedó en la primera posición y ya quedó ordenado. Ahora hay que repetir el procedimiento para ordenar el resto de los números, buscando el menor valor entre los desordenados es decir que ahora consideramos el vector para la búsqueda del menor partiendo desde el segundo valor ya que el primero está ordenado y se repite el mismo procedimiento en el segundo paso quedará ordenado el 3 y será intercambiado por el 6

2	3	10	8	6

Así se debe seguir el procedimiento siempre buscando el menor en un conjunto de datos que cada vez será más chico hasta lograr tener todo ordenado. La cantidad de repeticiones total es decir la cantidad de veces que debemos buscar el mínimo va a depender del tamaño del vector si el vector es de 5 elementos se debe buscar el mínimo 4 veces ya que cuando queden solo dos elementos al recuperar el mínimo el último quedará automáticamente ordenado.

Llevemos ahora dicho algoritmo a una función donde para la determinar la posición del menor valor del conjunto se utiliza una segunda función.

```
void OrdenSeleccion (int v[], int tam)
{
    int i,pmin, aux;
    for (i=0;i<tam-1;i++)
    {
        pmin = PosMinimo (v,i,tam);
        if (pmin!=i)
        {
            aux = v[i];
            v[i] = v[pmin];
            v[pmin] = aux;
        }
    }
}

int PosMinimo (int v[], int ini, int tam)
{
    int i, posMin;
    posMin = ini;

    for (i=ini+1;i<tam;i++)
        if (v[i]<v[posMin])
            posMin =i;

    return posMin;
}
```

En el código de la función puede verse que el ciclo se hace restando 1 ya que último quedará ordenado como se mencionó anteriormente. La determinación de la posición del mínimo dentro del vector a ordenar se delega en una función que recibe el vector, desde donde buscar el mínimo (para saltear los ya ordenados) y la cantidad de datos del vector. También puede notarse que solo se hace el intercambio si en la primera posición del vector desordenado (dado por la variable i) no se encuentra ya el menor valor.

En el caso de que se desee ordenar en forma descendente (de mayor a menor) el único cambio será utilizar una función que devuelva la posición del máximo dentro del vector en lugar de la posición del mínimo.

5.2 Ordenamiento por Burbujeo

Otro método de ordenamiento fácil de recordar es el ordenamiento por burbujeo que lo que hace es ir comparando posiciones adyacentes y haciendo intercambios entre dichas posiciones.

Por ejemplo, para ordenar de menor a mayor el mismo vector visto anteriormente, el algoritmo comienza comparando las dos primeras posiciones

compara

10	6	2	8	3

Y como el 10 es más grande que el 6 entonces hace el intercambio ya que buscamos que quede siempre el más chico primero.

6	10	2	8	3

Luego sigue comparando, pero desde la siguiente posición es decir compara ahora el 10 con el 2

6	10	2	8	3

Y como también el 2 es menor entonces intercambia

6	2	10	8	3

Se sigue con el mismo procedimiento, pero comparando ahora la 3er posición del vector con la cuarta

6	2	10	8	3

Y como el 10 es mayor que el 8 se debe realizar el intecambio

6	2	8	10	3

Se sigue comparando ahora las dos últimas posiciones

6	2	8	10	3

Y también se debe intercambiar quedando este vector

6	2	8	3	10

Como puede apreciarse el algoritmo luego de recorrer el vector una vez comparando posiciones adyacentes lo que hace es llevar a la última posición el valor más grande. Es decir que con este recorrido solo se ordenó un valor. Por lo tanto, hay que repetir el procedimiento para ordenar los 4 primeros valores ya que el último quedó ordenado.

El código de algoritmo es el siguiente:

```
void OrdenBurbujeo (int v[], int tam)
{
    int i,j, aux;
    for (i=0;i<tam-1;i++)
    {
        for (j=0;j<tam-1-i;j++)
            if (v[j]>=v[j+1])
            {
                aux = v[j];
                v[j] = v[j+1];
                v[j+1] = aux;
            }
    }
}
```

El ciclo externo indica la cantidad de veces que hay que recorrer el vector para que quede ordenado y el interno es el que controla la cantidad de comparaciones que hay hacer en cada recorrido. Nótese que al ciclo interno se le resta el índice del ciclo externo, esto se debe a que en cada recorrido hay un dato más que ya queda ordenado en las últimas posiciones del vector y por lo tanto cada vez se hacen menos comparaciones ya que la parte desordenada del vector es más chica.

En el caso de que se desee ordenar en forma descendente (de mayor a menor) el único cambio será cambiar el signo de comparación para realizar el intercambio si el dato de la posición j es menor al dato de la posición j+1.

5.3 Ordenamiento por Burbujeo optimizado

El algoritmo visto anteriormente puede mejorarse de dos formas:

- Detectando si el vector se llega a ordenar antes de realizar todos los recorridos
- Mirando hasta donde esta ordenado el vector ya que nos pueden entregar un vector parcialmente ordenado.

Para utilizaremos el siguiente algoritmo:

```
void OrdenBurbujeoOptimizado (int v[], int tam)
{
    int i,desordenado, aux,limite=tam-1;
    do
    {
        desordenado =0;
        for(i=0;i<limite;i++)
        {
            if (v[i]>v[i+1])
            {
                aux = v[i];
                v[i] = v[i+1];
                v[i+1] = aux;
                desordenado = i;
            }
        }
        limite =desordenado;
    }while (desordenado);
}
```

Este algoritmo implementa las dos mejoras, en lugar de un doble ciclo for el ciclo externo se maneja con un ciclo do while, para recorrer el vector y si no se realizaron intercambios en todo el recorrido quiere decir que ya estaba ordenado y por lo tanto no hace falta volver a recorrerlo.

El límite del ciclo interna se maneja con una variable que inicial recorre hasta el final del vector, pero para ciclos siguientes dicho límite será la posición del último intercambio ya que si al recorrer una vez no se hicieron intercambios luego de dicha posición era porque el resto del vector ya estaba ordenado.

5.4 Ordenamiento de vectores paralelos

Cuando se tienen vectores paralelos al ordenar un vector se debe también realizar el intercambio en el o los vectores que guardan información relacionada. Por ejemplo, si se tienen dos vectores paralelos uno con DNI de alumnos y otro con las notas y si busca mostrar un listado ordenado por nota, si los intercambios solo se realizan sobre el vector de notas quedarían mezclados los DNI de los alumnos con una nota que no les corresponde. Al ordenar vectores en paralelo el algoritmo se aplica sobre el vector que se quiere ordenar, pero al momento de realizar el intercambio se lo hace también, sobre los vectores relacionados.

```
void Ordenar (int vo[], int vi[], int ce)
{
    int i,j, aux;
    for (i=0;i<ce-1;i++)
        for (j=0;j<ce-1-i;j++)
            if (vo[j] < vo[j+1])
            {
                aux = vo[j];
                vo[j] = vo[j+1];
                vo[j+1] = aux;

                aux = vi[j];
                vi[j] = vi[j+1];
                vi[j+1] = aux;
            }
}
```

En este caso se necesita una única variable auxiliar ya que los vectores son del distinto tipo, si fueran de distinto tipo de dato entonces se necesitarían dos variables como auxiliares para realizar el intercambio.

6. Búsqueda binaria

La búsqueda secuencial es lenta ya que recorre una a una todas las posiciones del vector y si el dato a buscar está en las últimas posiciones entonces tiene que pasar primero por todo el resto. Existe un tipo de búsqueda mucho más rápida que es la búsqueda binaria pero que trabaja buscando sobre datos ordenados. Es decir que para poder realizar una búsqueda binaria es necesario que la colección esté previamente ordenada.

La búsqueda binaria trabaja reduciendo el conjunto de búsqueda a la mitad con cada lectura. Los pasos para desarrollar el algoritmo son los siguientes:

1. Sabiendo la cantidad de datos de la colección se calcula el dato central
2. Se lee el dato central de la colección
3. Se compara el dato buscado con el leído y según si es mayor o menor ya se descarta la mitad de la colección. Por ejemplo, si se busca el número 500 y el registro central tiene el número 300 se sabe que en la primera mitad de la colección no se va a encontrar el dato ya que está ordenada de menor a mayor. Por lo que ahora se reduce la búsqueda a la mitad siguiente de la colección.
4. Sino se encuentra se vuelve a calcular la mitad de la colección donde sí podría estar el dato y se repite el procedimiento hasta que se encuentre o hasta que se detecte que no se encontró. Esto se detecta porque los índices de búsqueda inicial y final se cruzan.

A continuación, se muestra el código de la función de búsqueda binaria sobre un vector considerando que previamente se encuentra ordenado de menor a mayor.

```
int BusquedaBinaria (int v[], int buscado, int ce)
{
    int  pos =-1, li=0, ls=ce-1, med;
    while (pos==-1 && li<=ls)
    {
        med=(li+ls)/2;
        if (v[med]==buscado)
            pos =med;
        else
            if (buscado>v[med])
                li = med+1;
            else
                ls = med-1;
    }
    return pos;
}
```